

CSCI 370 Final Report

Quantum Rings

Jose Abuan
Melody Goldanloo
Paul Lintzen
Aradhana Ramesh

Revised June 21, 2026



CSCI 370 Summer 2026

Prof. Donna Bodeau

Table 1: Revision history

Revision	Date	Comments
Original	May 24, 2026	Completed Sections: I. Introduction II. Functional Requirements III. Non-functional Requirements IV. Risks V. Definition of Done XIII. Team Profile - References - Appendix A – Key Terms
Rev – 2	May 31, 2026	Updated Sections: II. Functional Requirements III. Non-functional Requirements IV. Risks Completed Sections: VI. System Architecture
Rev – 3	June 7, 2026	Updated Sections: VI. System Architecture Completed Sections: VII. Software Test and Quality VIII. Ethical Considerations
Rev - 4	June 14, 2026	Updated Sections: IV. Risks VII. Software Test and Quality Completed Sections: IX. Project Completion Status X. Future Work XI. Lessons Learned
Rev - 5	June 17, 2026	Updated Sections: V. Definition of Done VI. System Architecture - References Completed Sections: XII. Acknowledgements
Final	June 21, 2026	Updated Sections: All Sections I – XIII, References, Appendix

Table of Contents

I. Introduction.....	3
A. Project Background	3
B. Project Objective and Scope	3
C. Client and Motivation.....	3
D. Previous Work and Existing Platform.....	3
E. Source of Data	3
F. Hardware and Backend Interface	3
G. Definitions and Abbreviations	4
H. Stakeholders, Users, and Maintenance	4
II. Functional Requirements	4
III. Non-Functional Requirements	5
IV. Risks	5
A. Project Risks	5
B. Product Risks	6
V. Definition of Done.....	6
VI. System Architecture.....	6
A. Process Diagram.....	6
B. Flow Diagram	7
C. Risk Register	8
D. List of Technical Issues	9
VII. Software Test and Quality.....	9
VIII. Project Ethical Considerations	11
A. ACM Principle 2: CLIENT AND EMPLOYER.....	11
B. ACM Principle 3: PRODUCT	11
C. ACM Principle 7: COLLEAGUES	11
D. IEEE Principle: I.....	11
IX. Project Completion Status/Results.....	12
A. Features Not Implemented	12
B. Performance testing results	13
X. Future Work	17
XI. Lessons Learned	17
XII. Acknowledgments.....	17
XIII. Team Profile	18
References.....	18
Appendix A – Key Terms.....	19
Appendix B – Feature Information	20

I. Introduction

A. Project Background

Quantum Simulation is an essential tool for testing quantum circuits before running them on physical quantum hardware, which is especially challenging when access to that hardware is limited. The resources needed to simulate a circuit can vary widely depending on the circuit's size, depth, gate structure, entanglement, and simulation settings. This makes simulation cost much harder to estimate than QPU execution, where pricing is primarily based on the number of shots or circuits. A relatively simple circuit may finish quickly, while a circuit with the same number of qubits but deeper structure or more complex entanglement can take much longer and require significantly more resources.

B. Project Objective and Scope

This project focuses on building a cost estimator for Open Quantum's new cloud simulation tool using the Quantum Rings simulator. Open Quantum currently offers a unified platform for access to multi-vendor Quantum Computers. They now want to expand to also offering cloud quantum simulation on classical hardware. The goal of our project is to estimate the resources required to run a given OpenQASM circuit before execution. Specifically, the system takes a QASM file and backend configuration, including backend type, precision, and threshold, and return estimates for runtime, peak memory usage, and recommended compute tier. These estimates can help provide users with more transparent pricing and help the platform allocate simulation resources more effectively.

The scope of this project includes generating benchmark data, extracting useful circuit and backend features, training a predictive model, and, as a stretch goal, packaging the final estimator into a deployable Python module or command-line tool. The project does not include building the full Open Quantum service or replacing the Quantum Rings simulation backend. Instead, the estimator is intended to support the existing simulation workflow by providing fast predictions before a job is submitted.

C. Client and Motivation

The client for this project is Quantum Rings. Quantum Rings provides the simulation backends used in this project and is supporting Open Quantum's expansion from QPU-only execution to on-demand cloud simulation. The client needs this estimator because simulation cost depends heavily on the structure of the circuit being run. Without an upfront estimate of runtime and memory usage, it's difficult to provide transparent pricing or choose an appropriate compute tier before execution.

D. Previous Work and Existing Platform

This project builds on ideas from the MIT iQuHACK 2026 Quantum Rings challenge, where participants used a provided dataset of circuit performance data to predict threshold and duration. This field session project differs from that challenge in several important ways. Instead of using strictly the provided static data, the team must generate benchmark data by running circuits directly. The model must also predict memory usage, treat threshold as an input rather than an output, and be packaged as a deployable estimator rather than a short-term hackathon solution.

E. Source of Data

The primary source of data for this project was benchmark results collected by running diverse OpenQASM circuits on Quantum Rings simulation backends. Our final circuit dataset was sampled from a variety of open-source repositories. The chosen circuits varied in size, depth, gate types, and entanglement patterns. Each benchmark run recorded the circuit, backend configuration, wall-clock runtime, and peak memory usage. This data was used to train and evaluate the estimator.

F. Hardware and Backend Interface

The estimator was trained using benchmark data collected from Quantum Rings simulation backends. The in-scope backends are Amber, Scarlet, and Serin, which represent CPU, GPU, and hybrid execution respectively. Each has its own use case where it performs best. The estimator used backend type, precision, and threshold as part of its input configuration since simulation performance depends on the selected backend.

G. Definitions and Abbreviations

Term	Definition
QASM	Quantum Assembly Language; A text-based format used to describe quantum circuits.
OpenQASM	An open standard/version of QASM used to represent quantum circuits.
QPU	Quantum Processing Unit; The physical quantum hardware used to run quantum circuits.
Quantum Rings	The project client and provider of the simulation platform/backends used to collect benchmark data.
Backend	The simulation engine/configuration used to run a circuit.
Threshold	A simulation setting that affects the tradeoff between accuracy and performance.
Duration	The total real-world time it takes for a simulation job to finish.
Compute tier	The recommended resource level for running a simulation job.

H. Stakeholders, Users, and Maintenance

The intended users of the software are Open Quantum customers who want to estimate the cost of a simulation job before running a quantum circuit. They are largely researchers and industry professionals testing new circuits. Additional stakeholders include Quantum Rings, Open Quantum platform maintainers, and the project team developing the estimator. After delivery, Quantum Rings will be responsible for maintaining and integrating the estimator into the broader simulation service.

II. Functional Requirements

A deployable inference model that takes an OpenQASM circuit file and a Quantum Rings backend configuration (backend type, precision, threshold) as inputs to predict upfront simulation compute costs.

Inputs:

- OpenQASM circuit file (.qasm)
- Quantum Rings backend and configuration metadata

Outputs (Predicted Metrics):

- Estimated peak memory usage
- Estimated duration

Implementation Constraints: Quantum libraries must rely on standard tools (e.g., quantumrings, qiskit). Machine learning libraries are restricted to common frameworks such as scikit-learn and xgboost. Common open-source software must be restricted to standard data science libraries. Note that any obscure or outside OSS dependencies require explicit client approval prior to integration.

Stretch Goal: If we get a reasonably accurate predictor trained and approved by client standards. A stretch goal would be to have a functional API that can connect our model to OpenQuantum and QuantumRings backend for ease of use on their actual product.

III. Non-Functional Requirements

Performance - An Ideal target inference latency must be less than 100 ms execution time per prediction under normal load. In the worst-case, maximum latency ceiling must be less than or equal to 2 to 3 seconds execution time under heavy algorithmic load or complex circuit scaling. Any prediction exceeding this ceiling was considered a performance failure.

Scalability and Extensibility - The codebase configuration must allow developers to profile and integrate new simulation backends seamlessly without rewriting the core inference engine pipeline. The inference model must scale to handle resource predictions across all three in-scope backends independently or concurrently.

Cost Efficiency & Resource Management - Because simulating deeply entangled random circuits can run for hours and consume significant memory, the cost estimator itself must be lightweight so it doesn't incur the very expenses it is trying to predict.

IV. Risks

A. Project Risks

Time Management – As we are working on a 5-week timeline we need to ensure we stay on track so we can start getting results and have time to revise and improve. Getting behind will carry over and throw off later weeks.

- **Likelihood:** Likely
- **Impact:** Moderate
- **Risk Mitigation Plan:** Use scrum board and actively communicate. Work proactively to get tasks done early and stay ahead.

Limited Quantum Knowledge – As we don't have a strong background in the quantum world, figuring out what aspects of a circuit are accurate predictors of performance will take considerable research. If we fail to extract accurate features our model will not be able to learn meaningful information and will produce useless predictions.

- **Likelihood:** Unlikely
- **Impact:** Major
- **Risk Mitigation Plan:** Consult with subject matter experts and utilize online resources and past projects to get an understanding of what features are most important. Additionally, we will perform testing with feature importances to figure out what features our model is relying on most heavily.

Overreliance on LLMs – As quantum computing is still very much in its infancy, reliable data is hard to find. Even the best LLMs today consistently make false statements regarding quantum topics and still have significant gaps in their knowledge, especially with more complex operations.

- **Likelihood:** Unlikely
- **Impact:** Moderate
- **Risk Mitigation Plan:** We will be very mindful of any information we get from LLMs and always fact check against reliable sources.

Lack of Reliable Training Data – We are relying on open source QASM files; thus, the availability of high-quality circuits may be limited. So, our results on our held-out set may still be substantially different from what real world cutting edge and advanced circuits look like.

- **Likelihood:** Likely
- **Impact:** Moderate

- **Risk Mitigation Plan:** We will use as much real-world data as possible and work closely with our client to ensure our data is well structured and as representative as possible. We also hope to use some of the client's stored circuits to get a good source of many varied circuits.

B. Product Risks

Poor Predictions –

- Time:** If our model severely underestimates the runtime, users may be left with much higher costs than expected after running their circuits which could cause major issues. On the other hand, if our time predictions are too high, users may opt to use another service despite the true cost being lower.
 - Memory:** If our model gives overly high memory predictions, we may allocate resources inefficiently and charge users for more resources than they need. On the other hand, if our model undershoots memory predictions processes may be slowed down considerably and take much longer than expected. In the worst case, they may fail completely.
- **Likelihood:** Moderate
 - **Impact:** Major
 - **Risk Mitigation Plan:** We will use as much data as we have available to train the model to get and get it's predictions as accurate as possible. We will also use a holdout test set to ensure we can a realistic measure real world performance. We will be transparent about our accuracy and the range of our predictions so that users understand the shortcomings of the model. We will also not deploy the model unless our accuracy is up to a certain standard as misleading customers is no help.

V. Definition of Done

The project will be considered complete when the team has delivered a working simulation cost estimation tool that accepts an OpenQASM file and a selected Quantum Rings backend configuration as input, then returns estimated duration, peak memory usage, and a recommended compute tier. At minimum, the model should support the main in-scope configuration parameters including the backend type, threshold, and precision settings, and should be tested on representative circuits from outside the training set.

The final deliverable will include a labeled benchmark dataset of circuit runs, documented feature extraction methods, trained and evaluated model(s), evaluation metrics, and a clear explanation of the model's accuracy and limitations. The team will also provide enough documentation for the sponsor or a future team to understand how the dataset was generated, how the model was trained and how new benchmark data or backends could be added later.

A polished interface wrapper, such as a command-line tool or deployable Python package will be treated as a stretch goal rather than a required completion item. The project will be considered done once the core dataset, model, evaluation, documentation, and sponsor-facing presentation are complete and reviewed.

VI. System Architecture

A. Process Diagram

Figure 1 models the broad overview of our design process. We input a variety of QASM files to extract quantum computing related features, runtimes, and peak memory. This dataset will come from a carefully selected subset of available QASM files. We want to make sure the dataset is representative of the complexity of modern-day computing research.

We will then conduct data analysis on these features to extract meaningful trends. This will include data cleaning, various visualizations, plotting distributions between features, identifying correlations between variables, and noting outlier detection. These patterns will then be used to fit various machine learning models to predict the runtimes and peak memory of QASM files. We will be fine tuning our models and selecting our final model to showcase.

Our model should output predicted runtimes and peak memory for user imputed QASM files. Ideally the model can help recommend an ideal compute tier for users to test on for best results.

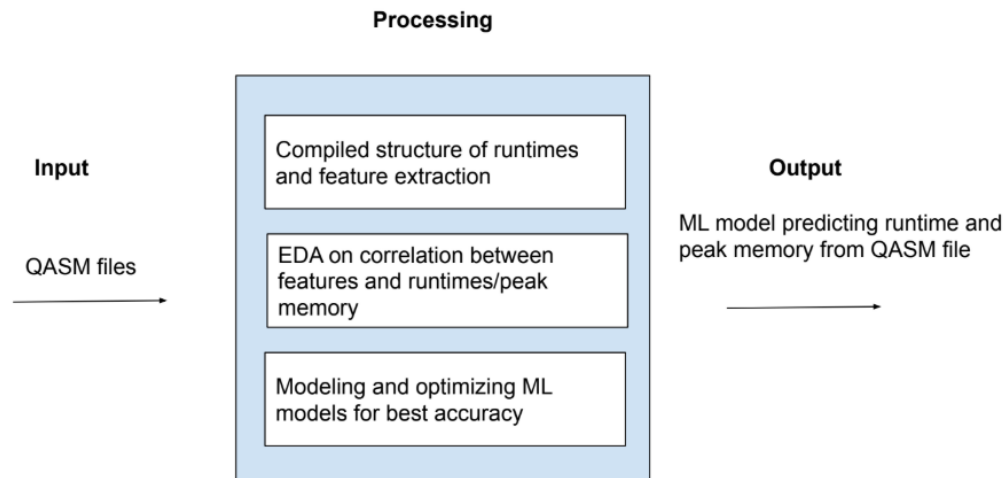


Figure 1: Process Diagram Representing Design Process

B. Flow Diagram

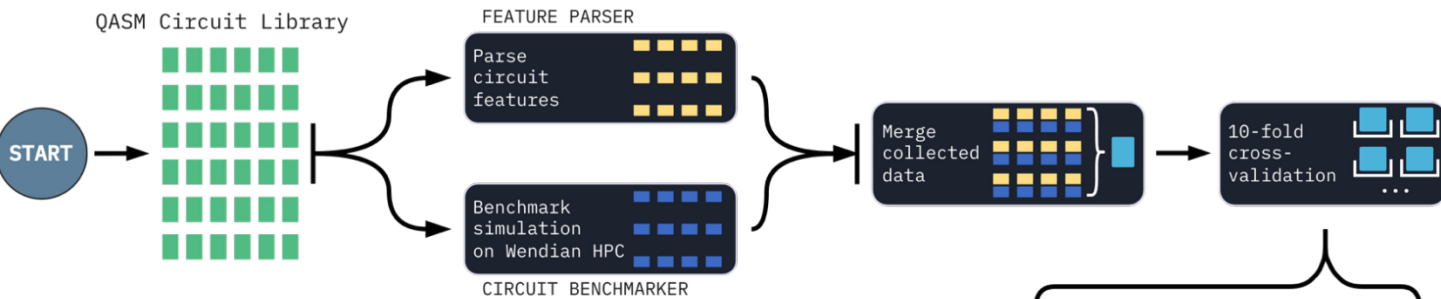
Figure 2 walks us through the general flow of our process for this project. We will begin by compiling a large set of OpenQASM circuits from a variety of online sources. Once we have a sufficiently large dataset we will run it through the Quantum Rings simulator with a variety of settings. As this is incredibly compute intensive especially for more complex files, we will be running this on Mines High Performance Compute Cluster (HPC), Wendian. Using this server is what makes this computation possible in our timeframe. For the user selected settings, there are three different backends to choose from: Scarlet, Amber and Serin corresponding to CPU, GPU, and hybrid. Then we choose our precision from double or single. Additionally, we will test a variety of threshold settings from 2-1024 corresponding to how accurate the final simulation is to the true result. Finally, we will set shots though for our project this will be a constant 1. Shots are just how many times we run the simulation. Since quantum circuits are probabilistic rather than deterministic more shots mean we get a better average of the true behavior.

Then for the additional information we will be recording; First is the hardware where we record the CPU, how many cores are allocated to the process, and how much ram we allocate. Additionally, for GPU runs we record the GPU, the GPU memory and the memory frequency as this often has a large impact on throughput. For the simulation specific results, we record the runtime of the circuit, the memory used as recorded by a background polling thread and finally the memory as recorded by an internal function. All of this will be stored in runs.json.

Simultaneously, we will build a feature parser that extract useful features from the raw QASM file that our model can train on. These are stored in features.json. We merge these two on file name to create our final dataset. At this point, we split off a handful of files completely to test our final model. Then we perform 10-fold cross validation with our remaining train data to get an accurate idea of performance numbers. After we get these results, we train the model one final time on the full train set. When evaluating the results of the cross validation, our performance target is an $R^2 > 0.95$ or above and we want this to be relatively consistent across all runtimes. To then evaluate the final trained model, we will test it on our held out files which were sampled to include low, medium, and high runtime runs. Users will enter their preferred settings and upload their QASM file, and the stored model will spit out memory and runtime

estimate to give an accurate price prediction upfront. If the model is not yet ready, we will examine the possible causes and return to either add new features or test a new model/tune hyperparameters.

MODEL DESIGN



FINAL PIPELINE

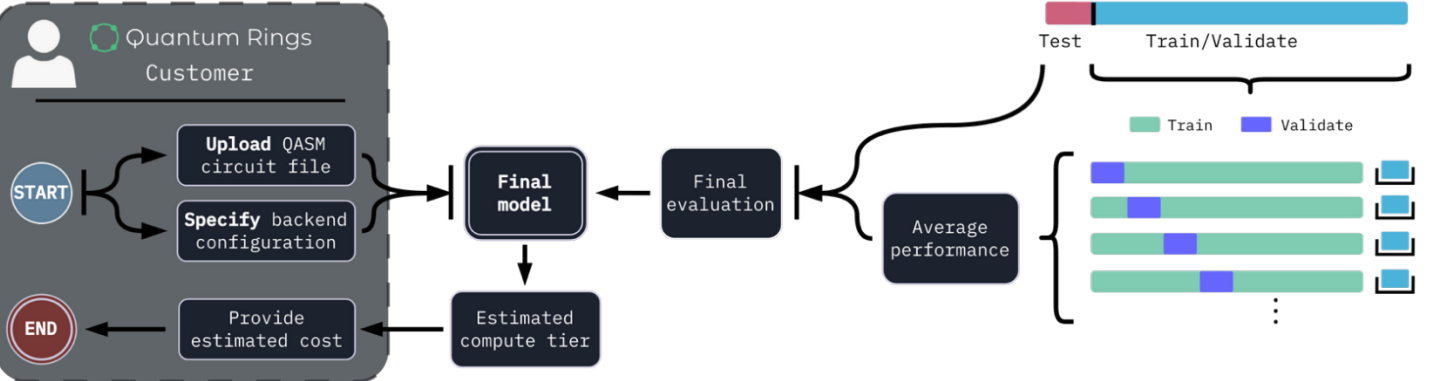


Figure 2: Flow Diagram Representation of ML Model

C. Risk Register

Asset (numbers)	Vulnerability	Threat	Likelihood	Impact	Risk	Treatment	Control	Mitigated Risk
Quantum Rings Dataset	Sensitive Data	Leaked files	Low	Catastrophic	8	Avoidance	Never share the company's files	Files are kept private between the team and company
ML Model	Final product should not be revealed	Leaked details on the specifics of the ML model	Low	Catastrophic	8	Avoidance	Do not share the final model details to anyone but the company and field session class(Company approved)	Company's property isn't stolen

Prediction Accuracy	Poor Predictions cost clients real money	Over or underestimate costs	Likely	Medium	8	Reduce Likelihood	Run various models and on various subsets	We will fine tune our model for best results
API Keys	Our API keys are personal to us and should not be published	If uploaded directly with code these could be compromised	Medium	Serious	9	Reduce Likelihood	Store API keys locally in .env and load with dotenv package.	API keys never publicly exposed and each user keeps theirs locally.

Table 1: Project Risks

As our project is largely building a model and compiling data, we don't have the same types of risks as in more user-oriented software engineering projects. Instead, our risks are largely concentrated around the security of the data we are generating and the impacts of the results.

D. List of Technical Issues

1. The runtimes of the QASM files are a point of conflict since our time is capped at 4 hours. We may need to get extended time if we want to run large QASM files. So far to address runtime issues we've relied on parallelizing work and splitting up large tasks. We may also implement checkpointing in future.
2. Our parser originally wasn't optimized well enough to extract all the 80+ features but our final parser was able to include all of them.
3. We need diverse subsets for representative data and ensure QASM file complexity. As we aren't quantum experts and the field is constantly evolving it is incredibly hard to build a truly representative set that can generalize to modern circuits being run by researchers.
4. We need a range of hardware to make this really be applicable for a large range of users but right now we are only using the student nodes in Wendian which have two specific CPUs and one type of GPU. Ideally, we will utilize collab along with our company's consumer GPU server as well to make the dataset more diverse.
5. As Quantum Rings is a small startup we have run into some issues with their SDK and needed to assistance with addressing some bugs before continuing.

VII. Software Test and Quality

Quality Plan:

Elements of Software Design	Implementation	Aspects of Code Quality
Code Reviews	Team members verify each other's code to ensure we are benchmarking (CPU/GPU statistics) and extracting QASM features correctly.	We focus on coding standards, error detection, and verifying results.
Resource Utilization	We are monitoring various resources (e.g. memory usage, CPU usage, etc.) for each QASM circuit. These statistics were fed to our models.	We want to be able to detect outliers in our data and verify expected trends.

	Team members verify and compare tracking. Additionally, EDAs allow us to detect outliers between various settings.	
Integration Testing	We are constantly updating the model as more parameters are generated through an automated system (i.e. As additional features are being parsed, they are being fed into the model).	We want to ensure that the pipeline is a cohesive system. We are verifying model accuracy as data is being fed in. We want to make sure our models' performances aren't deteriorating as new features are extracted.
Model Validation	We are constructing various models and are fine-tuning each model. We compare them with each other to submit the best model to the client.	We are verifying the implementation of each model. Additionally, we compare each metric of the model (i.e. Accuracy, MSE, etc.).
User acceptance testing	We present our final model to the client.	We are verifying our coding standards and validating that our model satisfies the client.

Table 2: Quality Plan

Sample Users Tests:

Test Name	User Story Description	Category	Acceptance Criteria (Test Scenario)	Expected Result
ML Test	As a machine learning analyst, I want a robust automated profiling pipeline to reliably execute thousands of quantum circuit variations so that I have uncorrupted baseline data for predictive modeling.	Telemetry & Data Collection	Execute a large-scale automated cluster array job over a diverse circuit dataset; verify telemetry (runtime, internal/external memory) is captured with zero unhandled script crashes.	100% of non-OOM runs are recorded; corrupted lines or network timeouts are gracefully skipped and logged without stopping the pipeline.
Feature Test	As a data analyst, I want the pipeline to parse quantum circuit files and extract key structural features (qubit count, gate depth, operation mix, etc.) so they can be used as predictive machine learning variables.	Feature Engineering	Pass a diverse set of validation QASM files through the circuit parser and compare the output vector against known circuit properties.	Features are correctly extracted and matched to execution logs, forming a structured matrix for model training.
Money Test	As a paying customer, I want a predictive engine that forecasts simulation runtime within a tight error margin so that I can plan allocations and justify future compute funding.	Prediction Engine Accuracy	Evaluate the trained regression model against an unseen verification dataset of quantum circuits.	The Mean Absolute Percentage Error (MAPE) for predicted vs. actual runtime falls within the acceptable target threshold (e.g. < 15%).
Researcher Test	As a researcher, I want the system to accurately predict runtime and compute requirements before a job is	Risk Mitigation & Boundary Testing	Input a set of range of different complexity circuits to to test prediction accuracy for	The system is able to accurately predict runtime and memory use for an input circuit

	sent to the cluster so that I know costs ahead of time and can choose the correct compute tier for my use case.		many possible circuit designs.	and gives the user a good idea of what compute tier is ideal for them.
Server Admin Test	As a server administrator, I want the prediction model to adjust its estimates based on hardware specifications (allocated RAM, CPU model) so that I can assign tasks efficiently to available nodes.	Cross-Platform Calibration	Train and test the engine using data from varying hardware baselines (e.g., varying core allocations or CPU generations).	The model accurately shifts its runtime coefficients when switching hardware profiles, maintaining low prediction error across environments.

Table 3: Sample User Tests

VIII. Project Ethical Considerations

Pertinent ACM/IEEE Principles:

A. ACM Principle 2: CLIENT AND EMPLOYER

"Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest."

We have a responsibility to Quantum Rings and to users to be ethical and honest about our final model. We must acknowledge any limitations in our final model (i.e. skews in our data, model accuracy, etc.). These issues directly impact pricing for the company's customers, so we must be transparent about our product.

B. ACM Principle 3: PRODUCT

"Software engineers shall ensure that their products and related modifications meet the highest professional standards possible."

Our model should meet the highest professional standards given the client's needs and restrictions. A major limitation in our model is the 5-week schedule. We must strive for high-quality work (i.e. debug code, documentation, etc.) despite these constraints.

C. ACM Principle 7: COLLEAGUES

"Software engineers shall be fair to and supportive of their colleagues."

It's crucial that our team stays accountable and respectful so we can complete this project. We want to ensure we are able to present our best work within the given time frame, the client's needs, and our own judgment.

D. IEEE Principle: I.

"To uphold the highest standards of integrity, responsible behavior, and ethical conduct in professional activities."

A significant part of this project requires us to receive feedback and build upon each prototype. We need to be honest with ourselves and the client about the limitations and errors in our work.

High Risk ACM/IEEE Principles:

ACM Principle 2: CLIENT AND EMPLOYER is one of the principles that is most in danger of being violated. If we aren't clear about the limitations of our final product, the machine learning model can inaccurately price customers and

recommend poor compute tiers. We must ensure that we are transparent to the company about the possible biases in our final model.

ACM Principle 3: PRODUCT is another crucial principle. It's essential that our client is satisfied with our final product. We want to adhere to coding standards, proper documentation, and present a viable model.

Michael Davis Tests:

Harm Test: Does our product benefit Quantum Rings customers?

Yes, as long as we abide by ethical engineering practices and acknowledge any limitations and biases in our final model, our project can be beneficial to others. Our model can help users gain confidence in the final pricing and help recommend a compute tier for ideal results.

Legality test: Would this choice violate a law or a policy of my employer?

If we presented our final product to the client, would there be any policy violations?

Our product does not violate a policy from Quantum Rings. We made sure to keep the client's QASM files private. Additionally, our GitHub repository is private to ensure confidentiality of the data, EDA, and model.

Ethical Consideration for Poor Software Implementation:

If our software quality plan is not implemented properly, Quantum Rings' users will likely face unpredictable and incorrect pricing estimates when running the software. Our model could also be recommending inadequate compute tiers, causing confusion among users as well. Furthermore, we want to ensure that company data stays private as the Quantum Rings profits from their database.

Security Considerations:

We have two security concerns. The Quantum Rings' private repository of QASM files should not be revealed. Additionally, the final model is confidential as predictive model is profitable to the client.

IX. Project Completion Status/Results

A. Features Not Implemented

Hybrid Backend with Expanded GPU Data

Due to infrastructure constraints, testing focused primarily on CPU execution with limited GPU coverage. Two factors contributed to this limitation:

- GPU execution was consistently slower than CPU for the circuit sizes tested, offering no performance advantage.

- Wendian does not permit splitting GPU nodes across concurrent tasks. Since each GPU node contains 4 GPUs, only 4 processes could run simultaneously, compared to 50 concurrent CPU processes. This disparity resulted in significantly less GPU data collected.

Multi-Shot Runtime Modeling

All runtime data was collected using single-shot execution. In practice, users typically run approximately 1,000 shots per circuit to obtain statistically reliable results. Before any production deployment, the model should be retrained and validated using realistic shot counts.

Dataset Representativeness

Of the 790 circuits collected, the majority had runtimes under 2 seconds, reflecting the early-stage nature of the quantum computing field where users are predominantly testing simple circuits. As the field matures, the composition of the dataset should be reviewed carefully, and ideally curated with input from a domain expert in quantum computing. See table two for more details.

Runtime Window	Runs (N)	MAE (seconds)	RMSE (seconds)	R ² (log)
0-1s	9099	0.10	0.48	0.2014
1-10s	5075	1.37	15.99	0.5711
10-100s	586	17.53	64.56	-0.4722
100-500s	288	38.86	101.96	-0.7117
500-1000s	34	269.58	410.82	-41.7858
1000-2000s	81	723.70	1218.01	-41.1013
2000-4000s	29	1580.32	1904.15	-159.4022
4000-6000s	19	1950.73	2659.31	-190.2303
6000-8000s	36	1943.45	3151.94	-180.9612
8000-10000s	25	1828.63	2635.09	-114.6283
10000-12000s	10	6912.75	7846.30	-6717.2056
12000-14000s	9	10362.18	11198.35	-19465.7059

Table 4: Runtime by Buckets

B. Performance testing results

Model Results

As mentioned above our final dataset consisted of 790 unique circuits. We tested these across 2 precision settings and 10 thresholds ~16,000 unique runs. We also tested across two backends being CPU and GPU. However, the GPU data is incomplete as there was no speedup for any tested circuits and hardware limitations already limited the throughput for our GPU runs. Our final model performed well in predicting our targets for lower runtime circuits though there was a clear drop off in performance for higher runtime files as there was simply not enough training data. We can see this is true by first looking at our runtime distribution in Figure 3.

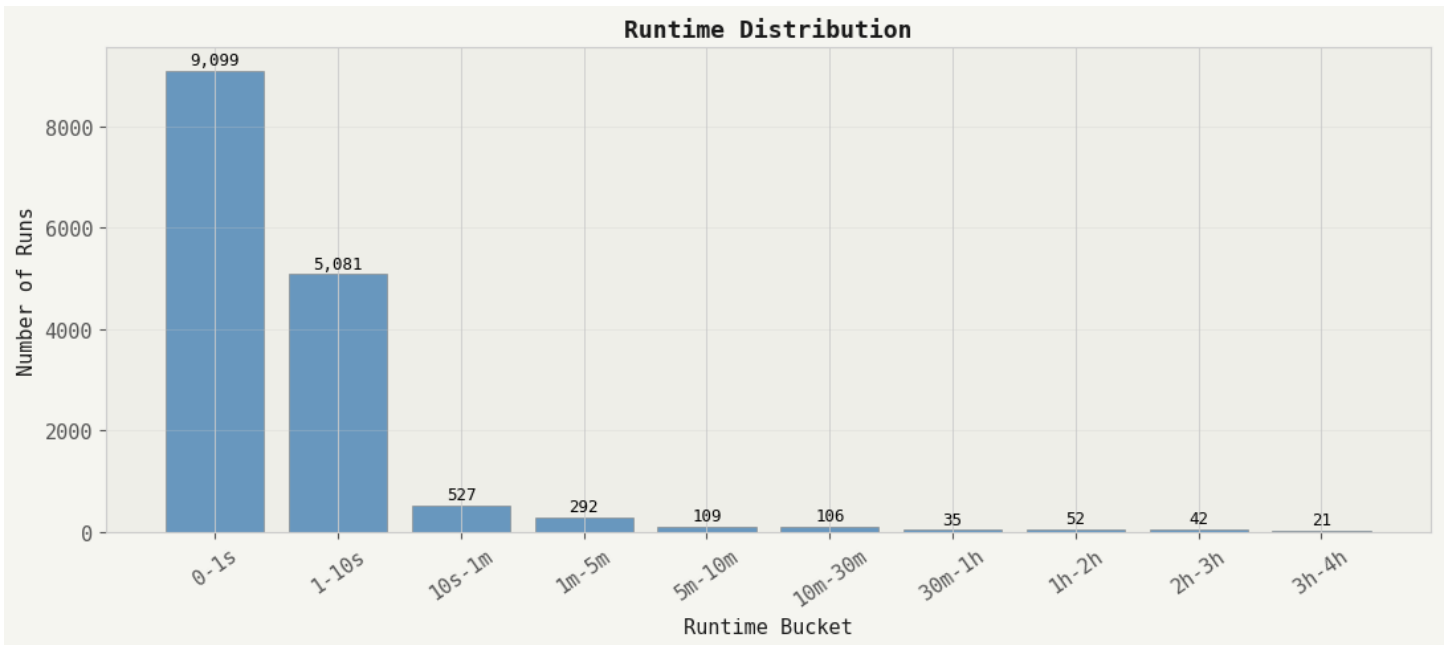


Figure 3: Runtime Distribution

With over 9000 of our configurations running in under 1 second and only 115 in the one to four hour range our data has an extreme skew. This is clearly reflected in our final model as can be seen later.

To ensure our model's recorded statistics are valid and we are not overfitting to the training data, we were careful to avoid leaking test files. In getting our result statistics we use 10-fold cross validation and calculated individual R^2 scores for each fold and then averaged them out. This means we split our data into 10 batches and trained it 10 times each time holding out only 1 batch. Additionally, in splitting our data we split on file name rather than just the run since the same file with different settings may still have similar performance. The figures below give an overview of our final model's performance. Figure 4 shows the final R^2 scores for predicting each of the three targets; runtime, peak program memory as recorded by a background polling thread and peak simulation memory as recorded by a built-in function. We can see that while the overall R^2 scores are good, we have a tendency to underpredict high runtime files. In a real world setting this could be catastrophic as we would severely underestimate the cost and either leave the user with a huge, unexpected bill or be forced to take a loss if we guarantee our price upfront.

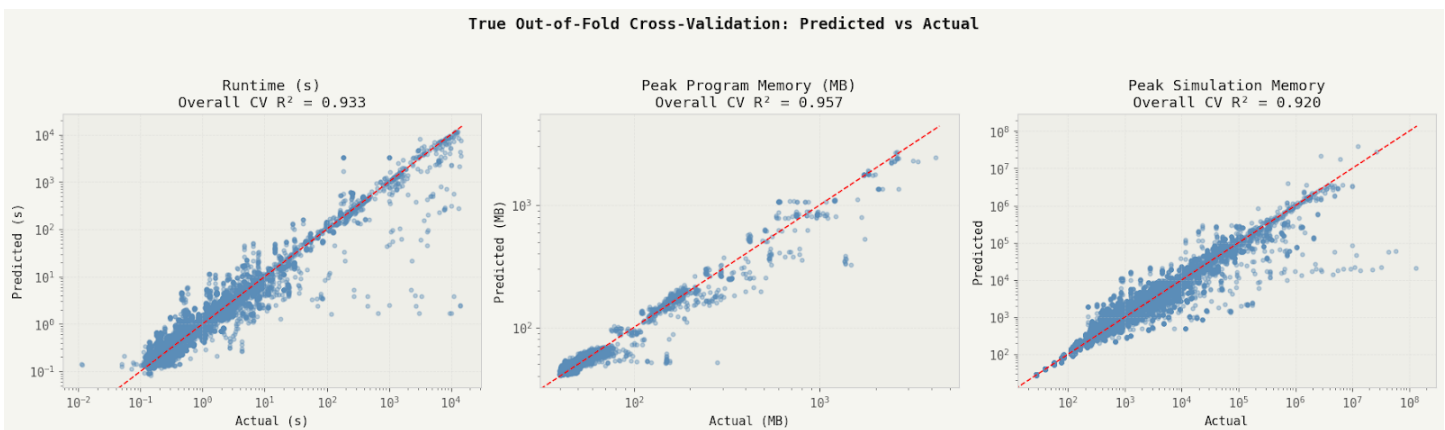


Figure 4: Final 10-fold CV R^2 Scores

To better see the aforementioned issue with predicting higher runtime circuits. Figure 5 is our model results broken up into runs under 10 minutes and over 10 minutes. Under 10 minutes we have an R^2 of 0.937 which overall means we do a

good job predicting the true runtime. However, for runs over 10 minutes our R^2 becomes negative meaning our model is worse than just predicting the average runtime value across all the runs in that range.

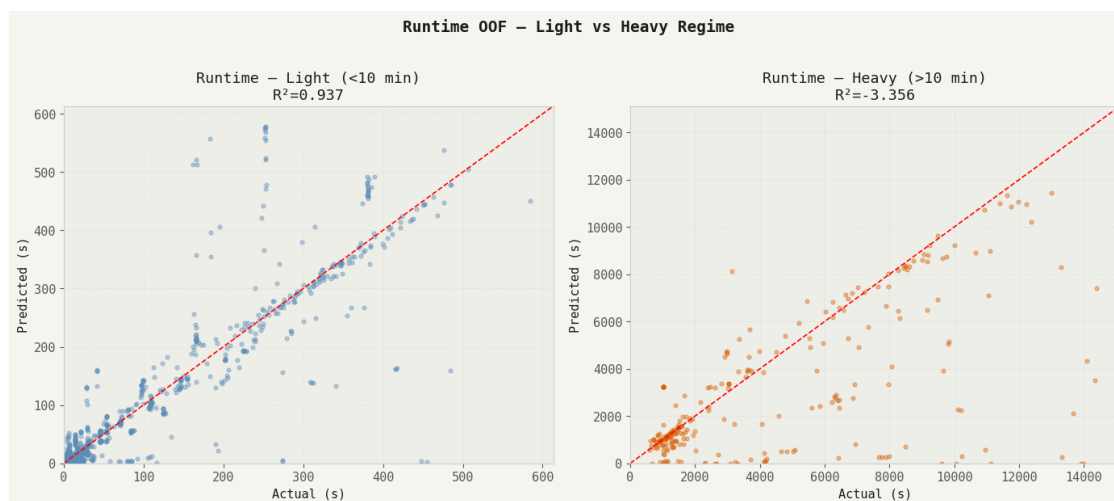


Figure 5: Split runtime results

Figure 6 shows the results of a slight variation of the model. Here we trained a model that instead tried to predict the 2.5% percentile, median, and 97.5% percentile. The idea behind this is to give a 95% confidence interval so users have more than just a number and can better understand how accurate the estimates are. We can again see that for lower runtime files our model performs reasonably well with a tight confidence interval. However, as we get into higher runtime configurations performance drops off drastically. We see a confidence interval that spans nearly the entire range of possible runtimes. From the grey bars showing amount of samples at each runtime we can see that these higher runtime areas clearly don't have enough data.

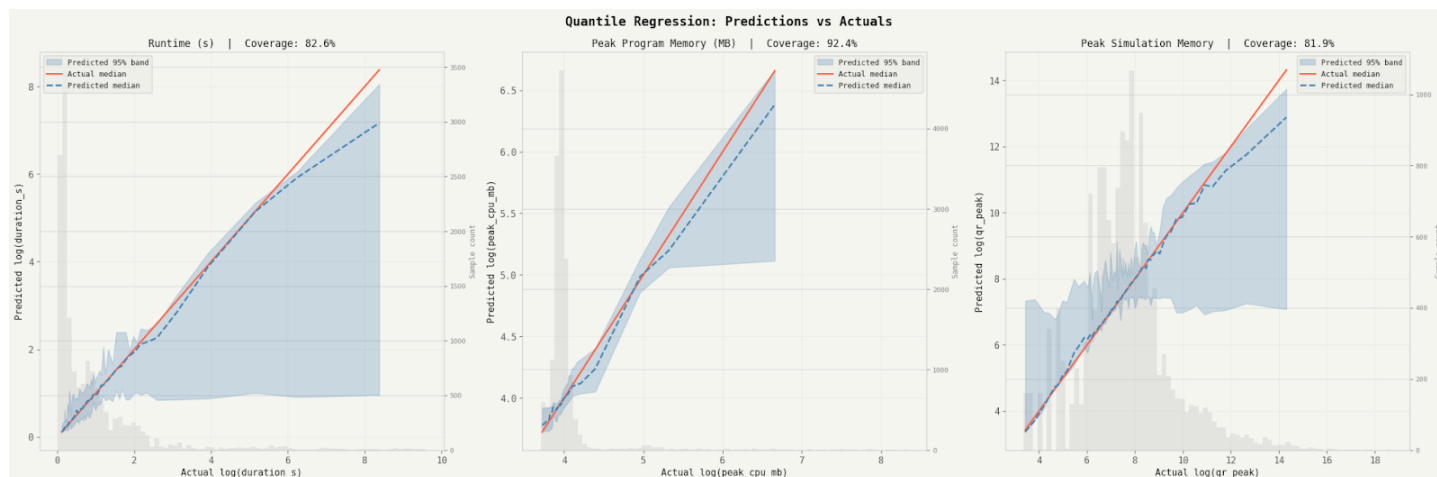


Figure 6: Quantile Model Results

Demo User Interface:

To show how our model would be used by an end user, we created simple flask app to test our model with new files. The application was easy to use with clear buttons and instructions for the user. This is not something our client will actually use and is more a proof of concept to demonstrate how the final UI may look and demonstrate the model's performance. Figure 7 shows an image of that interface after uploading a QASM file that the model has never seen and shows the true runtime results on the side. To process an uploaded file, our feature extractor first runs and calculates all the relevant features and the user's selected run configuration (threshold + precision). This is then fed to our stored

model which spits out a runtime estimate and peak program and simulation memory. Additionally, we feed the same data to our quantile model to give a range of prediction values. We can see that for this file with a runtime under 10 seconds our model does reasonably well and while the actual value is not correct our 95% confidence interval contains the true value and our memory result is very close.

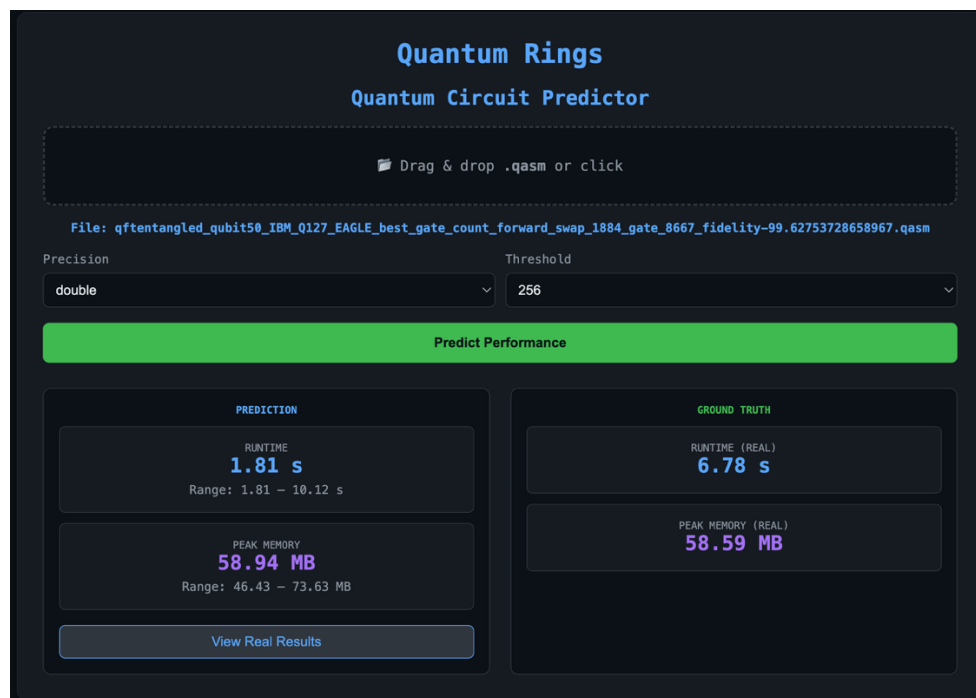


Figure 7: Short Runtime Flask App Demo

However, if we look at Figure 8 where we instead test a file with a much longer runtime the runtime results are much worse. Interestingly, the memory prediction remained pretty good though still worse than for the shorter runtime file.

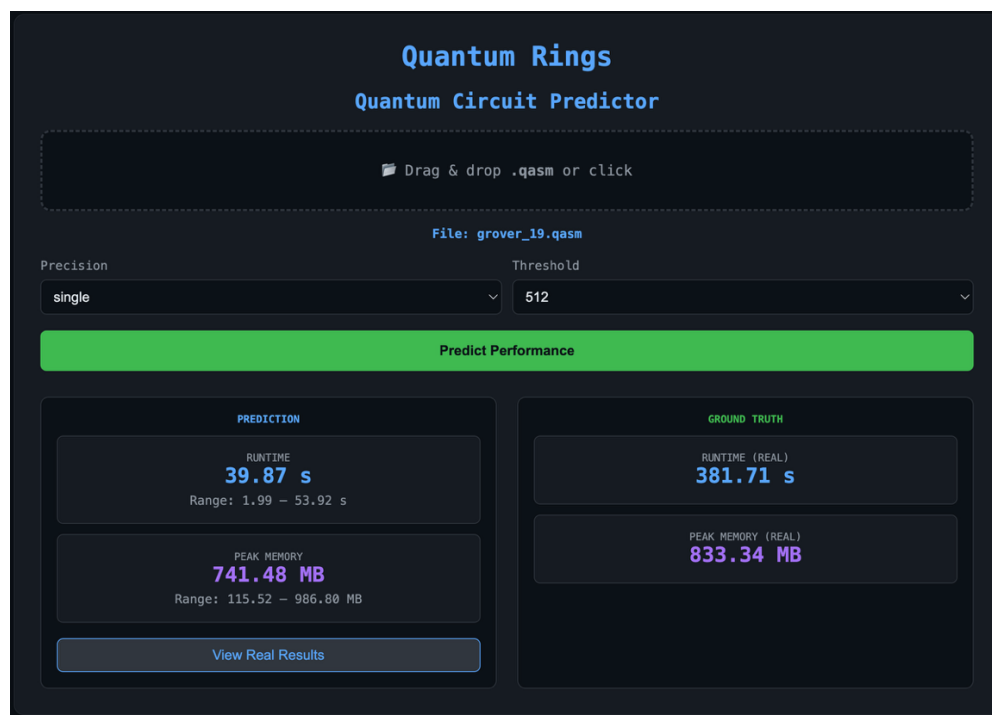


Figure 8: Long Runtime Flask App Demo

X. Future Work

This project has lots of possible future work that could be done, especially as the Quantum field evolves further. Below we have included a list of just a few that we feel may be most impactful.

- Include more circuits in the dataset. These circuits should be representative of real-world quantum research.
- Focus on finding more complex circuits with higher runtime to even out the distribution.
- Possibly integrate the parser with Quantum Rings existing data collection so that users generate lots of training data for a potential future model.
- Experiment with various shot settings rather than just 1.
- Test on more hardware as real-world cloud compute will not all have one model of processor and GPU.
- Resolve remaining issues with GPU backend performance.
- Include Hybrid backend testing.
- Test each circuit with varying amounts of memory and perform an analysis on performance.
- Create a CLI for the project.
- Improve the user interface for these models.

XI. Lessons Learned

Through this project we gained a lot of valuable experience in fields we had previously not touched and also improved our understanding of the ones we had.

We learned how to navigate and fully utilize a high-performance compute cluster through Mines Wendian. Benchmarking our tasks was an integral part of our project and we spent days just tweaking different runtime setting and setups with SLURM to optimize our benchmarking. Running these circuits proved to be computationally intensive, even when running in parallel and we had to ensure each process still had sufficient RAM. Individual processes on Wendian's student nodes also have a hard 4 hour runtime cap so we had to be creative with how we set up our benchmarking to avoid needing to wake up every 4 hours at night.

Additionally, our team gained experience and background in the quantum computing field. We needed a basic understanding of circuit format, QASM features, and the industry to understand the importance of our project. As we worked to benchmark performance and train our model we had to increasingly dive deeper into the underlying performance drivers of quantum execution and simulation.

Finally, we learned a lot about machine learning and data science. We experimented with various machine learning models for the best results and continuously tried different approaches attempting to improve our model. The choices we made for the models came from lots of exploratory data analysis and testing.

XII. Acknowledgments

We would like to thank the team at Quantum Rings: Omar Armbruster, Bob Wold, and Venkateswaran Kasirajan for sponsoring this project, providing access to the Quantum Rings SDK and simulator, and offering guidance and assistance throughout the field session. We would also like to thank Professor Donna Bodeau for her instruction and feedback throughout CSCI 370. We would also like to thank Master Student Noah Braunfeld for verifying our QASM circuit library. Finally, we thank the Colorado School of Mines IT Department for providing access to the Wendian High-Performance Compute cluster, which made benchmarking at the scale required for this project possible.

XIII. Team Profile



Jose Abuan

Senior

B.S. in Computer Science

Hometown: Aurora, CO

Interests: Robotics, Software Development, and Data Analysis



Melody Goldanloo

Senior

B.S. in Computer Science, M.S. in Quantum Engineering

Hometown: Lakewood, CO

Interests: Artificial Intelligence, Quantum Computing, Data Analysis, and Software Engineering



Paul Lintzen

Senior

B.S. in Computer Science, M.S. in Data Science

Hometown: Santa Cruz, CA

Interests: Machine Learning, Data Analysis, Networking, and Software Development



Aradhana Ramesh

Junior

B.S. in Computer Science

Hometown: Highlands Ranch, CO

Interests: Machine Learning, Software Development, and Robotics

References

- [1] Association for Computing Machinery, "ACM Code of Ethics and Professional Conduct," 2018. [Online]. Available: <https://www.acm.org/code-of-ethics>
- [2] Institute of Electrical and Electronics Engineers, "IEEE Code of Ethics." [Online]. Available: <https://www.ieee.org/about/ethics>
- [3] A. W. Cross et al., "OpenQASM 3: A broader and deeper quantum assembly language," *ACM Transactions on Quantum Computing*, vol. 3, no. 3, pp. 1–50, 2022.
- [4] A. Javadi-Abhari et al., "Quantum computing with Qiskit," 2024. [Online]. Available: <https://github.com/Qiskit/qiskit>
- [5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, San Francisco, CA, 2016, pp. 785–794.
- [7] MIT iQuHACK 2026, "Quantum Rings Challenge." [Online]. Available: <https://github.com/iQuHACK/2026-Quantum-Rings>
- [8] Colorado School of Mines, "Wendian High Performance Quantum Computing Cluster." [Online]. Available: <https://rc-docs.mines.edu/pages/guidelines.html#student-partition-of-wendian-hpc-cluster>

- [9] Quantum Rings, "Quantum Rings SDK Documentation." [Online]. Available: <https://www.quantumrings.com>
- [10] J. Ruane, E. Kiesow, J. Galatsanos, C. Dukatz, E. Blomquist, and P. Shukla, "The Quantum Index Report 2025," MIT Initiative on the Digital Economy, Massachusetts Institute of Technology, Cambridge, MA, May 2025. [Online]. Available: <https://qir.mit.edu/benchmarking/>
- [11] Rigetti Computing, "Novera QPU." [Online]. Available: <https://www.rigetti.com/novera>

Appendix A – Key Terms

Term	Definition
<i>QPU</i>	<i>Quantum Processing Unit. The core computational processor of a quantum computer.</i>
<i>QASM File</i>	<i>Quantum Assembly Language File. A plain text script used to define quantum assembly language.</i>
<i>OpenQASM</i>	<i>An open standard/version of QASM used to represent quantum circuits.</i>
<i>Quantum Rings</i>	<i>The project client and provider of the simulation platform/backends used to collect benchmark data.</i>
<i>Backend</i>	<i>The simulation engine/configuration used to run a circuit.</i>
<i>Scarlet</i>	<i>The CPU backend in the Quantum Rings simulator.</i>
<i>Amber</i>	<i>The GPU backend in the Quantum Rings simulator.</i>
<i>Serin</i>	<i>The Hybrid backend in the Quantum Rings simulator.</i>
<i>Duration</i>	<i>The total real-world time it takes for a simulation job to finish.</i>
<i>Compute tier</i>	<i>The recommended resource level/backend for running a simulation job.</i>

Appendix B – Feature Information

Feature	Category	Cost	Notes
cpu_model	backend	★☆☆☆☆	
threshold	backend	★☆☆☆☆	0–1024
precision	backend	★☆☆☆☆	single or double
platform	backend	★☆☆☆☆	0/1 (CPU/GPU)
backend	backend	★☆☆☆☆	(scarlet/amber/serin)
os	backend	★☆☆☆☆	
peak_mem	backend	★★★☆☆	
n_qubits	circuit size	★☆☆☆☆	
log_n_qubits	circuit size	★★☆☆☆	
n_cbits	circuit size	★☆☆☆☆	
n_lines	circuit size	★☆☆☆☆	
n_nonempty_lines	circuit size	★☆☆☆☆	<i>n_lines</i>
n_total_gates	circuit size	★☆☆☆☆	
log_total_gates	circuit size	★★☆☆☆	
circuit_depth	circuit size	★☆☆☆☆	
log_circuit_depth	circuit size	★★☆☆☆	
gates_per_qubit	circuit size	★★☆☆☆	
depth_per_qubit	circuit size	★★☆☆☆	
gates_per_depth	gate ratio	★★☆☆☆	(parallelism)
n_1q_gates	gate count	★☆☆☆☆	
n_2q_gates	gate count	★☆☆☆☆	
n_3q_gates	gate count	★☆☆☆☆	
n_multi_qubit_gates	gate count	★☆☆☆☆	
ratio_1q_gates	gate ratio	★★☆☆☆	
ratio_2q_gates	gate ratio	★★☆☆☆	
ratio_3q_gates	gate ratio	★★☆☆☆	
multi_qubit_gate_density	gate ratio	★★☆☆☆	
weighted_gate_count	gate count	★★☆☆☆	
log_weighted_gate_count	gate count	★★☆☆☆	
n_cx	gate count	★☆☆☆☆	
n_cz	gate count	★☆☆☆☆	
n_cp	gate count	★☆☆☆☆	
n_h	gate count	★☆☆☆☆	

n_u2	gate count	★☆☆☆☆	
n_u3	gate count	★☆☆☆☆	
n_ccx	gate count	★☆☆☆☆	
n_rx	gate count	★☆☆☆☆	
n_ry	gate count	★☆☆☆☆	
n_rz	gate count	★☆☆☆☆	
n_x	gate count	★☆☆☆☆	
n_y	gate count	★☆☆☆☆	
n_z	gate count	★☆☆☆☆	
n_s	gate count	★☆☆☆☆	
n_t	gate count	★☆☆☆☆	
n_swap	gate count	★☆☆☆☆	
n_rzz	gate count	★☆☆☆☆	
n_parameterized_gates	parameterized	★★★★☆	
n_unique_parameters	parameterized	★★★★☆	
parameter_entropy	parameterized	★★★★☆	
rotation_gate_count	parameterized	★★★★☆	
clifford_gate_count	magic	★★★★☆	<i>inferred from Clifford density</i>
non_clifford_gate_count	magic	★★★★☆	<i>inferred from magic</i>
clifford_gate_density	magic	★★★★☆	
non_clifford_gate_density	magic	★★★★☆	<i>inferred from magic</i>
magic_score	magic	★★★★☆	
entanglement_gate_count	entanglement	★★★★☆	
entanglement_ratio	entanglement	★★★★☆	2q/total
entanglement_density	entanglement	★★★★☆	
entanglement_score	entanglement	★★★★☆	
entanglement_range	entanglement	★★★★☆	
interaction_graph_n_edges	interaction graph	★★★★☆	
interaction_graph_density	interaction graph	★★★★☆	
interaction_graph_max_degree	interaction graph	★★★★☆	
interaction_graph_min_degree	interaction graph	★★★★☆	
interaction_graph_mean_degree	interaction graph	★★★★☆	
interaction_graph_median_degree	interaction graph	★★★★☆	
interaction_graph_degree_std	interaction graph	★★★★☆	
interaction_graph_degree_entropy	interaction graph	★★★★☆	

interaction_graph_avg_clustering	interaction graph	★★★★☆	
interaction_graph_diameter	interaction graph	★★★★☆	
interaction_graph_avg_path_length	interaction graph	★★★★☆	
interaction_graph_degeneracy	interaction graph	★★★★☆	
interaction_graph_betweenness centrality	interaction graph	★★★★☆	
avg_2q_gate_span	interaction distance	★★★★☆	
max_2q_gate_span	interaction distance	★★★★☆	
std_2q_gate_span	interaction distance	★★★★☆	
span_gini	interaction distance	★★★★☆	
graph_bandwidth_natural	interaction distance	★★★★☆	
graph_bandwidth_min	interaction distance	★★★★☆	
graph_bandwidth_heuristic	interaction distance	★★★★☆	
graph_cutwidth_min	cut/width/congestion	★★★★☆	
graph_cutwidth_heuristic	cut/width/congestion	★★★★☆	
graph_treewidth_min_degree	cut/width/congestion	★★★★★	001: approx treewidth
graph_treewidth_min_fill	cut/width/congestion	★★★★★	001: approx treewidth
cut_pressure_entropy	cut/width/congestion	★★★★☆	
recursive_bisection_avg_cut	cut/width/congestion	★★★★☆	